

25. Template Method Design Pattern in Java

25.1 Introduction

The **Template Method Pattern** is a **behavioral design pattern** that defines the skeleton of an algorithm in a superclass but lets subclasses override specific steps of the algorithm without changing its structure.

25.2 Problem it Solves

Avoids code duplication by abstracting common logic into a base class while allowing specific steps to be customized by subclasses.

25.3 What is Template Method Pattern?

- **Base class** defines a template method
- Some methods in the template are abstract or hook methods
- Subclasses override those specific methods

25.4 Real-World Analogy

Think of a **recipe**: the general structure of making a dish is fixed (boil, fry, garnish), but each cook might customize ingredients or steps.

25.5 Structure

```
package com.mannemlokes;

public abstract class AbstractClass {
    public final void templateMethod() {
        stepOne();
        stepTwo();
        optionalStep();
    }

    protected abstract void stepOne();

    protected abstract void stepTwo();

    protected void optionalStep() {
    } // hook
}
```

25.6 Benefits

- Promotes code reuse
- Enforces algorithm structure
- Easy to vary behavior without altering algorithm

25.7 Implementation Steps

1. Define an abstract class with a template method
2. Include abstract or hook methods
3. Subclasses override specific steps

25.8 Examples

Example1: Data Export Template

```
package com.mannemlokessh.example1;

abstract class DataExporter {
    public final void export() {
        readData();
        transformData();
        writeData();
    }

    protected abstract void readData();

    protected abstract void transformData();

    protected abstract void writeData();
}

class CSVExporter extends DataExporter {
    protected void readData() {
        System.out.println("Reading data from DB for CSV...");
    }

    protected void transformData() {
        System.out.println("Converting data to CSV format...");
    }

    protected void writeData() {
        System.out.println("Writing data to CSV file.");
    }
}

class JSONExporter extends DataExporter {
    protected void readData() {
        System.out.println("Reading data from API for JSON...");
    }

    protected void transformData() {
        System.out.println("Converting data to JSON format...");
    }

    protected void writeData() {
        System.out.println("Writing data to JSON file.");
    }
}

public class Example1ExportClient {
    public static void main(String[] args) {
        DataExporter csv = new CSVExporter();
        csv.export();

        System.out.println();

        DataExporter json = new JSONExporter();
        json.export();
    }
}
```

Output:

```
Reading data from DB for CSV...
Converting data to CSV format...
Writing data to CSV file.
```

```
Reading data from API for JSON...
Converting data to JSON format...
Writing data to JSON file.
```

Example2: Game Initialization Template

```
package com.mannemlokes.example2;

abstract class Game {
    public final void play() {
        initialize();
        startPlay();
        endPlay();
    }

    abstract void initialize();

    abstract void startPlay();

    abstract void endPlay();
}

class Cricket extends Game {
    void initialize() {
        System.out.println("Cricket Game Initialized!");
    }

    void startPlay() {
        System.out.println("Cricket Game Started. Play it!");
    }

    void endPlay() {
        System.out.println("Cricket Game Finished!");
    }
}

class Football extends Game {
    void initialize() {
        System.out.println("Football Game Initialized!");
    }

    void startPlay() {
        System.out.println("Football Game Started. Play it!");
    }

    void endPlay() {
        System.out.println("Football Game Finished!");
    }
}

public class Example2GameClient {
    public static void main(String[] args) {
        Game game = new Cricket();
    }
}
```

```

        game.play();

        System.out.println();

        game = new Football();
        game.play();
    }
}

```

Output:

```

Cricket Game Initialized!
Cricket Game Started. Play it!
Cricket Game Finished!

Football Game Initialized!
Football Game Started. Play it!
Football Game Finished!

```

Example3: Web Framework Request Handling

```

package com.mannemlokes.example3;

abstract class WebRequestHandler {
    public final void handleRequest() {
        authenticate();
        process();
        log();
    }

    protected abstract void authenticate();

    protected abstract void process();

    protected void log() {
        System.out.println("Default logging.");
    }
}

class AdminRequestHandler extends WebRequestHandler {
    protected void authenticate() {
        System.out.println("Authenticating admin...");
    }

    protected void process() {
        System.out.println("Processing admin request...");
    }

    protected void log() {
        System.out.println("Logging admin access.");
    }
}

class UserRequestHandler extends WebRequestHandler {
    protected void authenticate() {
        System.out.println("Authenticating user...");
    }
}

```

```

        protected void process() {
            System.out.println("Processing user request...");
        }
    }

    public class Example3WebClient {
        public static void main(String[] args) {
            WebRequestHandler admin = new AdminRequestHandler();
            admin.handleRequest();

            System.out.println();

            WebRequestHandler user = new UserRequestHandler();
            user.handleRequest();
        }
    }
}

```

Output:

```

Authenticating admin...
Processing admin request...
Logging admin access.

Authenticating user...
Processing user request...
Default logging.

```

25.9 Template vs Strategy vs Factory Method

Pattern	Intent
Template Method	Define algorithm skeleton
Strategy	Select algorithm at runtime
Factory Method	Delegate object creation to subclasses

25.10 Pros and Cons

**Pros**

- Promotes code reuse
- Simplifies complex algorithms
- Makes common code stable and subclass code flexible

**Cons**

- Tight coupling between base and subclasses
- Inheritance is less flexible than composition

25.11 When to Use / Not to Use

Use When:

- You have multiple implementations of an algorithm
- Parts of the logic are shared across variants

Avoid When:

- You prefer composition over inheritance

- The variation is better represented as a Strategy

25.12 Summary

Feature	Description
Intent	Define invariant algorithm skeleton
Use Case	Framework hooks, template utilities
Key Benefit	Reuse common logic, customize variations

The **Template Method Pattern** is ideal when you want to enforce a consistent structure while allowing extensibility for specific steps.